

Aula HUD e MENU

Leonardo Lima Silva

Tá querendo programar um jogo, mas não tem paciência né, meu caro?

Mas na hora de assistir 5200 reels, 1350 TikToks e mais 4 vídeos sobre uma treta aleatória de dois famosos que não vai impactar absolutamente nada na sua vida... aí a preguiça some.

(Relaxa, isso também foi uma autocrítica.)

Então faz o seguinte: senta a bunda na cadeira aí.

Eu me chamo Leonardo, e hoje eu vou te ensinar a criar **Menus e HUD dentro da Godot usando GDScript**. Então, começando do zero, pra quem ainda não me conhece: eu sou estudante de Bacharelado Ciência da Computação da UNESP e faço parte do grupo Japonets.

E juntos a gente criou um curso chamado **CODEQUEST**, que ajuda você, jovem internauta, a dar os primeiros passos no sonho de programar seus próprios projetos.

Então chega de enrolação, e vamos falar de **menus de jogos**.

Menus são aquelas telas que aparecem antes ou durante o jogo.

Menu inicial, menu de pausa, menu de configurações...

Basicamente, eles são responsáveis por controlar a **navegação do jogador dentro do jogo**.

É por meio deles que o jogador pode iniciar o jogo, acessar configurações, voltar para o menu principal ou sair do jogo. Ou seja, o menu é meio que o **controle remoto do jogo**.

E sim... aquele menu que você ignora e sai apertando Enter até o jogo começar.

Agora vamos falar de uma outra coisa muito importante: a **HUD**. HUD significa **Heads-Up Display**(Não, eu não sei falar inglês). Traduzindo para a linguagem humana: são as informações que ficam aparecendo na tela **enquanto você está jogando**.

Por exemplo: barra de vida, pontuação, munição, mapa ou tempo. Se o menu é o controle remoto do jogo, a HUD é tipo o **painel do carro enquanto você dirige**. É ela que te mostra tudo que está acontecendo durante a partida.

Agora vamos trazer isso para dentro da Godot. Antes de sair criando botão aleatório na tela, tem uma coisa importante que você precisa entender sobre interface na Godot. Então acalma essa ansiedade, larga esse reels ou tiktok e presta atenção.

A Godot separa os elementos de interface dos objetos do jogo usando um tipo especial de nó chamado **Control**(CALMA, ABAIXA ESSAS ARMAS, EU VOU DESCOMPLICAR). Tudo que é interface: botão, texto, painel, barra de vida, normalmente deriva desse tipo de nó. A principal vantagem disso é que esses elementos se adaptam automaticamente à tela do jogador. Então se o jogador estiver em um monitor maior, menor ou até em um celular, a interface continua organizada.

E é por isso que a gente usa nós como:

Control
Button
Label
TextureRect
Containers

Porque eles ajudam a construir interfaces de forma organizada. Agora tem uma coisa que facilita muito a vida quando estamos criando interfaces: os **containers**. Containers são nós que organizam automaticamente os elementos filhos.

Por exemplo:

Um **VBoxContainer** organiza tudo na vertical.

Um **HBoxContainer** organiza tudo na horizontal.

Isso evita que você precise ficar posicionando cada botão manualmente na tela. Então no nosso menu principal eu vou usar um **VBoxContainer**, justamente para empilhar os botões de forma automática.

Agora vamos criar o nosso **menu principal**. Primeiro a gente cria uma nova cena. O nó principal pode ser um **Control**, já que estamos lidando com interface. Depois disso a gente adiciona um **VBoxContainer**, que vai organizar os botões verticalmente.

E agora vamos adicionar três botões básicos:

Play
Options
Exit

Com isso a gente já tem a estrutura básica de um menu(Perceba que o Elden ring tem o mesmo esquema de botões, conhecidencia?). Mas agora precisamos fazer esses botões realmente funcionarem.

Começando pelo botão **Play**.

Quando o botão Play for pressionado, a gente quer que o jogo saia do menu e carregue a cena principal do jogo. Para isso conectamos o sinal **pressed** do botão e usamos o seguinte código:

```
func _on_play_pressed():  
    get_tree().change_scene_to_file("res://scenes/game.tscn")
```

Basicamente o que isso faz é mandar a Godot trocar a cena atual pela cena do jogo. Ou seja, é aqui que a gameplay realmente começa. Agora vamos implementar o botão **Exit**, ou melhor, poderíamos deixar sem ele, assim ninguém nunca vai conseguir fechar jogo né?

```
func _on_exit_pressed():  
    get_tree().quit()
```

Quando esse botão for pressionado, o jogo simplesmente fecha. Agora que a gente já tem um menu funcionando, vamos falar rapidamente de outro tipo de menu que aparece em praticamente todo jogo: o **menu de pausa**.

A ideia é simples: enquanto o jogador está jogando, ele pode apertar uma tecla e o jogo pausa. E aí aparece um pequeno menu com opções como:

- Continuar
- Voltar para o menu
- Sair do jogo

Na Godot isso é bem simples de implementar. Primeiro a gente detecta quando o jogador aperta a tecla de pausa.

Quando isso acontecer podemos usar: `get_tree().paused = true`

Isso pausa praticamente toda a lógica do jogo. E então mostramos o menu de pausa na tela.

Quando o jogador quiser continuar jogando, basta fazer: `get_tree().paused = false`

E o jogo volta normalmente.

Agora vamos criar uma **HUD simples dentro do jogo**. A HUD normalmente fica por cima da cena do jogo, mostrando informações importantes para o jogador. Nesse exemplo a gente vai criar duas coisas básicas: pontuação e barra de vida.

Para isso criamos um **Control** dentro da cena do jogo que será responsável pela interface. Dentro dele podemos adicionar um **Label** para mostrar a pontuação e uma **barra de vida**, como um `TextureProgressBar`.

Agora vamos fazer a pontuação funcionar.

Primeiro criamos uma variável para armazenar o score:

```
var score = 0
```

E agora uma função para adicionar pontos:

```
func add_score(value):  
    score += value  
    $ScoreLabel.text = "Score: " + str(score)
```

Sempre que o jogador ganhar pontos, a gente chama essa função. Ela soma os pontos e atualiza o texto da HUD. Assim o jogador consegue ver sua pontuação sendo atualizada em tempo real durante o jogo. Agora tem uma coisa importante de entender: a HUD normalmente **não funciona sozinha**. Ela recebe informações do jogo.

Por exemplo:

- quando o jogador toma dano
- quando ganha pontos
- quando o tempo muda
- quando um objetivo é concluído

O jogo envia essas informações para a HUD, e a HUD atualiza os elementos da interface. Ai se você espere que eu te ensine a fazer tudo isso em um só vídeo...calma ai parceiro, vai assistir os outros videos de CODEQUEST.

E aqui vai uma dica muito importante para quem está começando. Um erro muito comum é tentar colocar elementos da HUD diretamente dentro da cena do jogador ou da câmera. Isso até pode funcionar no começo, mas rapidamente vira uma bagunça. O ideal é sempre separar a HUD em uma **cena própria de interface**, que apenas recebe informações do jogo. Isso deixa o projeto mais organizado e facilita muito quando você quiser adicionar novos elementos depois.

E pronto.

Agora você já sabe criar:

um **menu principal funcional**
botões que realmente executam ações
uma **HUD básica**
e até um **menu de pausa**

Tudo isso usando **Godot e GDScript**.

Se você gostou do vídeo, compartilha com aquele seu amigo que diz que quer fazer jogo... mas passa seis horas seguidas no TikTok.

E se quiser aprender mais sobre programação de jogos e desenvolvimento de projetos, dá uma olhada no **CODEQUEST**, que é o curso que a gente criou para ajudar quem está começando.

E é isso.

A gente se vê no próximo vídeo.